DATA WAREHOUSING & DATA MINING

B.Tech 3rd Year — 5th Semester

Complete Booklet Notes — All 5 Units

CO 1: Be familiar with mathematical foundations of data mining tools. (K1, K2)

How to use this booklet:

- Each Unit has its own colour theme so you can tell them apart at a glance.
- Every Unit starts with a SYLLABUS BOX — the exact topics you must cover.
- Inside each unit, topics are arranged from MOST important to LEAST important (exam-weightage order).
- Tables are used wherever a comparison or difference is asked in exams.
- Example boxes are added wherever an example makes the concept easier to remember.

CONTENTS

UNIT I : Introduction to Data Warehousing & Multi-Dimensional Data Model

SYLLABUS — What this unit covers

- Data Warehousing: Overview, Definition
- Data Warehousing Components
- Building a Data Warehouse
- Warehouse Database
- Mapping the Data Warehouse to a Multiprocessor Architecture
- Difference between Database System and Data Warehouse
- Multi-Dimensional Data Model
- Data Cubes
- Stars, Snow Flakes, Fact Constellations Schema

1. Data Warehouse — Definition and Overview

A Data Warehouse is a large, central storage of data collected from many different sources, kept mainly to help managers make better decisions. It is NOT used for day-to-day transactions — it is used for analysis and reporting.

The famous definition (by W.H. Inmon):

"A Data Warehouse is a Subject-Oriented, Integrated, Time-Variant and Non-Volatile collection of data in support of management's decision-making process."

The 4 keywords in this definition (Very Important):

Keyword	Meaning
Subject-Oriented	Data is organised around major subjects like Customer, Sales, Product — not around applications.
Integrated	Data is collected from many different sources (databases, files, systems) and made consistent (same naming, same units, same format).
Time-Variant	Data is stored with a time reference (e.g. last 5-10 years) so we can study trends over time.
Non-Volatile	Once data enters the warehouse, it is not changed or deleted. Only new data is added. Old data is kept as history.

Example

A bank collects data from its ATM system, loan system

Keyword	Meaning
and credit-card system. All this scattered data is cleaned, combined, and stored subject-wise (e.g. all data about 'Customer') in one place with date stamps — this combined store is the Data Warehouse.	

2. Difference between Database System (OLTP) and Data Warehouse (OLAP)

This is one of the MOST asked questions in exams. A normal database is built for OLTP (Online Transaction Processing) and a Data Warehouse is built for OLAP (Online Analytical Processing).

Point	Database (OLTP)	Data Warehouse (OLAP)
Purpose	Day-to-day operations (insert, update, delete)	Analysis, decision-making, reporting
Data	Current, detailed data	Historical, summarised data
Users	Clerks, cashiers, operators	Managers, analysts, executives
Design	Normalized (ER Model)	De-normalized (Star / Snowflake Schema)
Operations	Read + Write (frequent updates)	Mostly Read-only (queries)
Size	Small to medium (MBs to few GBs)	Very large (GBs to TBs)
Response time	Milliseconds (fast, small transactions)	Seconds to minutes (complex queries)
Orientation	Application-oriented	Subject-oriented

3. Data Warehousing Components

A Data Warehouse system is made up of the following main components/building blocks:

- Source Data (Overall) Component — data coming from production systems, internal office systems, external sources, and archived data.
- Data Staging Component — this is where ETL (Extraction, Transformation, Loading) happens. Data is pulled from sources, cleaned, converted, and made ready.
- Data Storage Component — the actual warehouse database where the cleaned and integrated data is stored (usually in Star/Snowflake schema).
- Information Delivery Component — delivers data to the end user through reports, queries, and OLAP tools.
- Metadata Component — 'data about data'. It stores information like where data came from, how it was transformed, and what it means.
- Management and Control Component — coordinates all the services within the data warehouse (scheduling, monitoring).

4. Building a Data Warehouse

There are two common approaches/methodologies:

Approach	Given by	Idea
Top-Down Approach	Bill Inmon	First build one big central Enterprise Data Warehouse, then create small Data Marts from it.
Bottom-Up Approach	Ralph Kimball	First build small Data Marts for each department, then combine them to form the warehouse.

Steps to build a Data Warehouse (general process):

- Requirement gathering (understand what the business wants to analyse)
- Design the data model (choose Star / Snowflake schema)
- Extract data from source systems
- Transform data — clean, integrate, convert units/formats
- Load data into the warehouse (ETL process)
- Build OLAP cubes / reporting layer
- Test, deploy, and maintain the warehouse

5. Warehouse Database

The Warehouse Database is the core engine that actually stores the warehouse data. It must be able to handle:

- Very large volumes of data (VLDB — Very Large Databases)
- Complex, ad-hoc queries involving joins/aggregations across many tables
- Bulk data loading (loading millions of rows at once during ETL) rather than single-row transactions
- Support for parallel query processing to give fast answers on huge data

Because of this, a special class of database engines is used, and RDBMS vendors add warehouse-specific features like bitmap indexing, materialized views, and parallel query execution.

6. Mapping the Data Warehouse to a Multiprocessor Architecture

Since a data warehouse handles huge data and heavy queries, a single processor is not enough. So the warehouse database is 'mapped' (spread out) across multiple processors to divide the work and finish queries faster. There are 3 common multiprocessor architectures:

Architecture	Full Form / Meaning	Key Point
SMP	Symmetric Multiprocessor	Many processors share ONE common memory and ONE common disk. Simple but has a limit on scaling.
MPP	Massively Parallel Processing	Each processor has its OWN memory and OWN disk (nothing shared). Scales very well for huge warehouses.
Cluster	Clustered Systems	A set of independent, loosely connected computers/nodes that work together and appear as a single system.

The data is usually divided using a technique called Partitioning, so that each processor works on its own chunk of data at the same time (parallelism), which greatly speeds up large queries.

7. Multi-Dimensional Data Model

A Data Warehouse uses a multi-dimensional model, unlike the flat 2-D tables of a normal database. Data is viewed as a Data Cube with Facts and Dimensions.

Term	Meaning	Example (Sales Warehouse)
Fact	The measurable, numeric data we want to analyse	Sales amount, Quantity sold
Dimension	The different angles/perspectives used to look at the fact	Time, Product, Location, Customer
Measure	A specific numeric value of the fact	Total Sale = 50,000
Fact Table	Central table holding facts + foreign keys of dimensions	Sales_Fact table
Dimension Table	Table holding descriptive details of a dimension	Product table (Product_ID, Name, Category)

8. Data Cubes

A Data Cube lets us view data in multiple dimensions at once. Even though it is called a 'cube' (3-D), it can actually have many more dimensions (n-dimensional) — the word 'cube' is just used for easy visualisation.

> ***Example***
> *A company wants to see Sales data by Time, Product and Location. This 3-dimensional view (Time x Product x Location) forms a Data Cube. Each cell of the cube stores the sales value for that combination, e.g., 'Sales of Shoes in Delhi in Jan 2026'.*

Common OLAP operations performed on a Data Cube:

Operation	What it does
Roll-up	Moves from detailed data to summarised data (e.g. from City level to Country level) — climbing UP a concept hierarchy.
Drill-down	Opposite of roll-up. Moves from summary to more detailed data (e.g. from Year to Month to Day).
Slice	Selects ONE value for one dimension and views the resulting sub-cube (e.g. Sales only for the year 2026).
Dice	Selects TWO or more dimensions and creates a smaller sub-cube (e.g. Sales for 2025-2026 AND for Product = Electronics).
Pivot (Rotate)	Rotates the data axes to give a different presentation of the data (rows become columns and vice-versa).

9. Schemas: Star, Snowflake, and Fact Constellation

These are the 3 ways to design/model the tables of a data warehouse.

(a) Star Schema

One central Fact table is connected directly to several Dimension tables. It looks like a star, hence the name. Dimension tables here are NOT further split (de-normalized).

(b) Snowflake Schema

It is an extension of Star Schema where dimension tables are further broken/normalized into sub-dimension tables. It looks like a snowflake with branches.

(c) Fact Constellation Schema (Galaxy Schema)

Here, MULTIPLE Fact tables share common Dimension tables. It is used when a warehouse deals with multiple business processes together (e.g. Sales and Shipping sharing the same Time and Product dimensions).

Feature	Star Schema	Snowflake Schema	Fact Constellation
Structure	Simple, single fact + dimensions	Fact + normalized dimensions (sub-tables)	Multiple fact tables + shared dimensions
Normalization	De-normalized dimension tables	Normalized dimension tables	Mixed
Query Speed	Fast (fewer joins)	Slower (more joins)	Depends on complexity

Feature	Star Schema	Snowflake Schema	Fact Constellation
Storage Space	More redundancy, more space	Less redundancy, less space	Depends on design
Complexity	Simple to understand	More complex to design	Most complex
Best used when	Simplicity and speed are priority	Storage space needs to be saved	Warehouse supports many subject areas

Example

Star: Sales_Fact linked directly to Time, Product, Store dimension tables. Snowflake: Product dimension is further split into Product table + Category table + Brand table. Fact Constellation: Sales_Fact and Shipping_Fact both use the same Time and Product dimension tables.

UNIT II : Data Warehouse Process and Technology

SYLLABUS — What this unit covers

- Warehousing Strategy
- Warehouse Management and Support Processes
- Warehouse Planning and Implementation
- Hardware and Operating Systems for Data Warehousing
- Client/Server Computing Model & Data Warehousing
- Parallel Processors & Cluster Systems
- Distributed DBMS Implementations
- Warehousing Software
- Warehouse Schema Design

1. Warehouse Planning and Implementation

This is the most practical, most-asked topic of this unit — the real-life steps followed to implement a data warehouse project.

- Step 1: Requirement Analysis — meet business users, understand what questions they want answered.
- Step 2: Data Modelling — design the Star/Snowflake schema based on requirements.
- Step 3: Architecture Design — decide hardware, software, network needed.
- Step 4: ETL Design — decide how data will be Extracted, Transformed and Loaded.
- Step 5: Development — build the warehouse, write ETL scripts, build cubes/reports.
- Step 6: Testing — check data accuracy, performance, and query results.
- Step 7: Deployment and Maintenance — go live, then monitor and maintain regularly.

Implementation is usually done in small, incremental phases (build one data mart at a time) rather than a single 'big-bang' rollout, because warehouse projects are large and risky.

2. Warehousing Strategy

Before building a warehouse, an organisation must decide its overall strategy — a roadmap of how the warehouse will be built and grown.

- Decide the scope: whether to build one Enterprise-wide warehouse first (Top-Down) or small Data Marts first (Bottom-Up).
- Decide priority subject areas (e.g. start with Sales, then add Finance later).
- Plan budget, timeline, and the team (business analysts, DBAs, ETL developers).
- Plan for future scalability — the warehouse should be able to grow as more data/subjects are added.

3. Warehouse Schema Design

Schema design means deciding how fact and dimension tables will be structured (recall Star/Snowflake/Fact Constellation from Unit 1). Good schema design directly affects query performance.

- Identify the Grain — the lowest level of detail stored in the fact table (e.g. 'one row per sale transaction').
- Identify Dimensions — all the 'by what' angles of analysis (Time, Product, Customer, Location).
- Identify Facts/Measures — the numeric values to be analysed (Sales Amount, Profit, Quantity).
- Build Hierarchies inside dimensions for roll-up/drill-down (e.g. Day → Month → Quarter → Year).

4. Warehouse Management and Support Processes

Once a data warehouse is running, ongoing management activities are needed to keep it healthy:

Process	Purpose
Load Management	Manages the regular loading of new data (daily/weekly ETL runs).
Query Management	Monitors and manages user queries so the system does not slow down.
Storage Management	Manages disk space, archiving of very old data, indexing.
Metadata Management	Keeps 'data about data' updated (source, meaning, transformation rules).
Service Level Management	Ensures the warehouse meets agreed performance/availability targets.

5. Hardware and Operating Systems for Data Warehousing

A data warehouse needs powerful, reliable hardware because of the huge data volume and heavy queries.

- High storage capacity — large, fast disks (often RAID configurations for safety and speed).
- Strong processing power — multiple CPUs/cores to support parallel query execution.
- Large RAM — to cache data and speed up repeated queries.
- Reliable, scalable Operating System — usually UNIX/Linux-based servers or Windows Server, chosen for stability with big workloads.

6. Client/Server Computing Model and Data Warehousing

In the Client/Server model, work is split between a Client (which requests data, e.g. a report tool on a user's PC) and a Server (which stores and processes data, e.g. the warehouse database server).

- 2-Tier Architecture: Client talks directly to the Data Warehouse Server.

- 3-Tier Architecture: Client → Application/OLAP Server (middle layer for business logic) → Data Warehouse Server. This is more common in real warehouses because it separates presentation, logic, and data.

7. Parallel Processors and Cluster Systems

(Builds on Unit 1's SMP/MPP/Cluster topic — here the focus is on how they help performance.)

System	How it Helps Warehousing
SMP (Symmetric Multiprocessing)	Good for medium warehouses; multiple CPUs share memory so simple to manage.
MPP (Massively Parallel Processing)	Best for very large warehouses; each node works independently on its own data partition, giving very high speed.
Cluster Systems	A group of connected servers that share the workload and also give fault-tolerance (if one node fails, others continue).

8. Distributed DBMS Implementations

A Distributed Database Management System (DDBMS) stores data across multiple physical locations/sites but the user feels like it is a single database.

- Data Fragmentation: dividing a table into smaller parts and storing them at different sites.
- Data Replication: keeping copies of the same data at multiple sites for reliability and faster local access.
- Distributed Query Processing: a single user query is broken down and executed across multiple sites, then results are combined.

Distributed DBMS concepts are used in data warehousing when data is geographically spread (e.g., different regional offices) but needs to be analysed together.

9. Warehousing Software

Different categories of software tools are needed to build and run a warehouse:

Category	Purpose	Example Type
ETL Tools	Extract, Transform, Load data from sources into the warehouse	Informatica, SSIS type tools
Warehouse DBMS	Stores the actual warehouse data	Oracle, Teradata, SQL Server type engines
OLAP Tools	Let users analyse data using cubes (Roll-up, Drill-down, etc.)	ROLAP/MOLAP/HOLAP tools
Reporting Tools	Create dashboards, charts, and reports for end users	BI/reporting tools

Category	Purpose	Example Type
Metadata Tools	Manage and document metadata	Metadata repositories

UNIT III : Data Mining: Preprocessing, Cleaning and Reduction

SYLLABUS — What this unit covers

- Data Mining: Overview, Motivation, Definition & Functionalities
- Data Processing, Forms of Data Pre-processing
- Data Cleaning: Missing Values, Noisy Data (Binning, Clustering, Regression, Computer & Human Inspection), Inconsistent Data
- Data Integration and Transformation
- Data Reduction: Data Cube Aggregation, Dimensionality Reduction, Data Compression, Numerosity Reduction
- Discretization and Concept Hierarchy Generation
- Decision Tree

1. Data Mining — Overview, Motivation, Definition and Functionalities

Data Mining is the process of discovering interesting, hidden, and useful patterns from a large amount of data. It is also called 'Knowledge Discovery in Databases (KDD)'.

Motivation (Why do we need Data Mining?):

- Huge amounts of data are generated every day, but most of it is unused — this is called 'data rich, but information poor'.
- Businesses need hidden patterns (e.g. buying trends) to make smart decisions and stay competitive.
- Manual analysis of huge data is impossible — automated pattern discovery is required.

Data Mining Functionalities (types of patterns that can be mined):

Functionality	What it Finds	Example
Classification	Assigns data into predefined categories/classes	Classifying an email as Spam or Not-Spam
Clustering	Groups similar data together (classes are NOT predefined)	Grouping customers with similar buying habits
Association Rule Mining	Finds items that occur together frequently	'People who buy bread also buy butter'
Prediction / Regression	Predicts a future numeric value	Predicting next month's sales
Characterization	Summarizes general features of a class of data	Summary of 'high-spending customers'
Outlier / Anomaly Analysis	Finds data that does not fit the general pattern	Detecting fraud transactions
Evolution Analysis	Studies trends of objects whose behaviour changes over time	Stock price trend analysis

2. Data Pre-processing — Forms

Real-world data is usually dirty — incomplete, noisy, and inconsistent. Data Pre-processing prepares raw data so mining algorithms give correct results. 'Garbage in, garbage out' — poor quality data gives poor quality mining results.

Form of Pre-processing	What it Does
Data Cleaning	Fills missing values, removes noise, fixes inconsistent data.
Data Integration	Merges data from multiple sources into one consistent store.
Data Transformation	Converts data into forms suitable for mining (normalization, aggregation).
Data Reduction	Reduces data size while keeping the same analytical results (fewer attributes/rows but same meaning).
Data Discretization	Converts continuous (numeric) data into discrete categories/intervals.

3. Data Cleaning

Data Cleaning routines work to fill in missing values, smooth out noise, identify outliers, and correct inconsistencies.

(a) Handling Missing Values

Method	Description
Ignore the tuple	Simply skip the row with missing value (used only when many values are missing in that row).
Fill manually	A human expert fills the missing value (slow, but accurate for small data).
Use a global constant	Fill all missing values with a label like 'Unknown' or 'N/A'.
Use attribute mean/median	Fill with the average or median value of that column.
Use mean/median of same class	Fill with the average value of that column, but only from rows of the same class.
Use the most probable value	Use methods like regression or decision tree to predict/estimate the missing value.

(b) Handling Noisy Data (Noise = random error or variance in a measured value)

Noise is smoothed out using these techniques:

Technique	Description
Binning	Data is sorted and divided into equal-sized groups called 'bins'. Each bin's values are then smoothed by bin mean, bin median, or bin boundaries.

Technique	Description
Regression	Data is fit into a regression function (line or curve) so that one attribute helps predict/smooth another.
Clustering	Similar values are grouped into clusters. Values that fall outside any cluster are treated as outliers/noise and can be removed.
Combined Computer and Human Inspection	Suspicious values are detected by a computer program and then verified/corrected by a human expert.

Binning in detail — 3 smoothing methods:

- Smoothing by bin means: replace every value in the bin with the bin's average value.
- Smoothing by bin medians: replace every value in the bin with the bin's median value.
- Smoothing by bin boundaries: replace each value with the closest boundary (minimum or maximum) value of the bin.

> ***Example***
> *Sorted data: 4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34 Bin 1: 4, 8, 9, 15 | Bin 2: 21, 21, 24, 25 | Bin 3: 26, 28, 29, 34 Bin-Mean smoothing → Bin 1 becomes: 9, 9, 9, 9 (mean of 4,8,9,15 = 9)*

(c) Inconsistent Data

This means data that has conflicts — e.g. same customer having two different addresses in two different systems, or a discount value that does not match the item's actual price. It is corrected using knowledge engineering tools or manual correction based on external references.

4. Data Integration and Transformation

Data Integration

Combines data from multiple sources (databases, files, etc.) into one single, coherent data store. Key issues faced:

- Schema Integration — matching different attribute names for the same real-world entity (e.g. 'cust_id' vs 'customer_number').
- Redundancy — same data appearing more than once (checked using correlation analysis).
- Detection and resolution of data value conflicts — same entity, different value in different sources (e.g. different units like kg vs pounds).

Data Transformation

Converts data into a form suitable for mining. Common strategies:

Strategy	Description
Smoothing	Removes noise from data (same as in data cleaning).
Aggregation	Summary or aggregation operations applied to data (e.g. daily sales → monthly sales).

Strategy	Description
Generalization	Low-level data is replaced by higher-level concepts using concept hierarchies (e.g. 'city' generalized to 'country').
Normalization	Attribute data is scaled to fall within a small, specific range, such as 0.0 to 1.0.
Attribute Construction	New attributes are constructed from the given ones to help mining (e.g. 'Area' constructed from 'Length' and 'Width').

Normalization formulas:

$$\text{Min-Max Normalization: } v' = [\,(v - \min)\,/\,(\max - \min)\,] \times (\text{new\_max} - \text{new\_min}) + \text{new\_min}$$

$$\text{Z-Score Normalization: } v' = (v - \text{mean})\,/\,\text{standard\_deviation}$$

5. Data Reduction

Data Reduction produces a smaller, reduced version of the data set that still gives (almost) the same analytical/mining results, but takes much less time and storage.

Technique	Description
Data Cube Aggregation	Data is stored in summarized/aggregated form using OLAP data cubes (e.g. yearly totals instead of daily records).
Dimensionality Reduction	Reduces the number of attributes/variables under study, e.g. using Attribute Subset Selection or techniques like Principal Component Analysis (PCA).
Data Compression	Encoding techniques (e.g. wavelet transforms) reduce data size. Can be Lossless (no information lost) or Lossy (some detail lost).
Numerosity Reduction	Replaces the original data with smaller alternative data representations, e.g. Regression models, Histograms, Clustering, or Sampling.

6. Discretization and Concept Hierarchy Generation

Discretization converts continuous, numeric attribute values into a small number of discrete intervals/labels, which are then used as if they were categorical data.

> ***Example***
> *Age (numeric): 12, 18, 25, 40, 65 → Discretized into categories: 'Child', 'Youth', 'Adult', 'Senior'.*

A Concept Hierarchy organizes attribute values into different levels of abstraction — from more detailed (low-level) to more general (high-level).

> ***Example***
> *Location Concept Hierarchy: Street → City → State → Country (Street is most detailed, Country is most general).*

- Discretization and concept hierarchies are very useful in OLAP for roll-up and drill-down operations (recall Unit 1).

7. Decision Tree

A Decision Tree is a flow-chart-like tree structure used both for Classification and for helping in Data Reduction/Pre-processing decisions. Each internal node represents a test on an attribute, each branch represents the outcome of the test, and each leaf node represents a class label.

- Root Node: the topmost node, represents the best attribute to split the data.
- Internal (Decision) Node: represents a test/condition on an attribute.
- Leaf Node: represents the final output/class label.
- Common algorithms to build decision trees: ID3, C4.5, CART (these use measures like Information Gain / Gini Index to decide the best attribute to split on).

> ***Example***
> *To decide 'Play Tennis or Not', the root node might test 'Outlook' (Sunny/Overcast/Rain); each branch leads further down to attributes like 'Humidity' or 'Wind', ending in leaf nodes labelled 'Yes' or 'No'.*

UNIT IV : Classification, Clustering and Association Rules

SYLLABUS — What this unit covers

- Classification: Definition, Data Generalization, Analytical Characterization
- Analysis of Attribute Relevance, Mining Class Comparisons
- Statistical Measures in Large Databases
- Statistical-Based, Distance-Based, and Decision Tree-Based Algorithms
- Clustering: Similarity/Distance Measures, Hierarchical & Partitional Algorithms (CURE, Chameleon)
- Density-Based Methods (DBSCAN, OPTICS)
- Grid-Based Methods (STING, CLIQUE)
- Model-Based Method — Statistical Approach
- Association Rules: Large Itemsets, Basic/Parallel/Distributed Algorithms, Neural Network Approach

1. Classification — Definition

Classification is a two-step process that predicts categorical (discrete, unordered) class labels for new data, based on a training set of already-classified data.

- Step 1 — Learning (Training) Step: A classification model/algorithm is built by analysing a training data set whose class labels are already known.
- Step 2 — Classification Step: The built model is used to predict class labels for new, unknown data.

Example

Training data of past loan applicants (with known label 'Approved'/'Rejected') is used to build a model. This model is then used to predict whether a NEW applicant's loan should be Approved or Rejected.

2. Data Generalization and Analytical Characterization

Data Generalization summarizes a large set of task-relevant data from a low level to a higher, more general conceptual level (using concept hierarchies), making the data easier to understand.

Analytical Characterization goes one step further — after generalizing data, it also analyses which attributes are actually relevant/important for the characterization, instead of blindly generalizing every attribute.

Steps in Analytical Characterization:

- Data collection (collect the task-relevant data using a query)
- Perform attribute relevance analysis (check which attributes matter)
- Perform generalization on the relevant attributes only
- Present the result (as generalized relation, cross-tab, chart, or rule)

3. Analysis of Attribute Relevance

Not all attributes are equally useful for a mining task. Attribute Relevance Analysis measures how relevant/important each attribute is, so that irrelevant attributes can be removed before mining — this improves both speed and accuracy of results.

- Common measures used: Information Gain, Gini Index, Correlation Analysis (Chi-square test for categorical data, correlation coefficient for numeric data).

4. Mining Class Comparisons

Class Comparison (also called Discrimination) compares the general features of a target class with one or more contrasting classes, to find what makes the target class different.

> ***Example***
> *Comparing features of customers who 'buy a laptop' (target class) versus customers who 'do not buy a laptop' (contrasting class) to find distinguishing patterns, e.g. income level or age group.*

5. Statistical Measures in Large Databases

Basic statistical measures help describe and summarize the data before mining:

Measure	What it Tells	Formula/Idea
Mean	Central/average value	Sum of values ÷ number of values
Median	Middle value when data is sorted	Middle value (or average of two middle values)
Mode	Most frequently occurring value	Value with highest frequency
Range	Spread of the data	Maximum value − Minimum value
Variance / Standard Deviation	How much data varies from the mean	Average of squared differences from mean (Std Dev = √Variance)

6. Classification Algorithms

(a) Statistical-Based Algorithms

Use probability theory to classify data. The most popular example is the Naïve Bayes Classifier, based on Bayes' Theorem, which assumes attributes are independent of each other.

$$\text{Bayes' Theorem: } P(H|X) = [\,P(X|H) \times P(H)\,] \,/\, P(X)$$

(b) Distance-Based Algorithms

Classify a new data point based on its distance/closeness to known points. The most popular example is the k-Nearest Neighbour (k-NN) Classifier — it finds the 'k' closest training points to the new point and assigns the majority class among them.

Euclidean Distance between two points: $d = \sqrt{[\,(x2-x1)^2 + (y2-y1)^2\,]}$

(c) Decision Tree-Based Algorithms

Build a tree structure (as explained in Unit 3) using measures like Information Gain (ID3), Gain Ratio (C4.5), or Gini Index (CART) to pick the best splitting attribute at each node.

7. Clustering — Introduction

Clustering is the process of grouping a set of data objects into clusters, such that objects within the SAME cluster are highly similar, but objects in DIFFERENT clusters are very dissimilar. Unlike classification, clustering is unsupervised — class labels are NOT known in advance.

Similarity and Distance Measures:

Measure	Formula
Euclidean Distance	$d(x,y) = \sqrt{[\ \Sigma\ (xi - yi)^2\ ]}$
Manhattan Distance	$d(x,y) = \Sigma\ \|xi - yi\|$
Minkowski Distance	$d(x,y) = [\ \Sigma\ \|xi - yi\|^p\ ]^{(1/p)}$ (general form of the above two)
Cosine Similarity	Measures the angle between two vectors (used often for text data)

8. Hierarchical and Partitional Clustering Algorithms

Type	Description
Partitional Clustering	Divides data into a fixed number (k) of non-overlapping clusters in one step. Example: k-Means, k-Medoids.
Hierarchical Clustering	Builds a tree (dendrogram) of clusters, either by merging small clusters into bigger ones (Agglomerative / bottom-up) or splitting a big cluster into smaller ones (Divisive / top-down).

Special Hierarchical Algorithms:

Algorithm	Key Idea
CURE (Clustering Using Representatives)	Instead of using just one point (centroid) to represent a cluster, it uses several well-scattered representative points. This helps it find clusters of unusual, non-spherical shapes and handle outliers better.
Chameleon	A hierarchical algorithm that measures similarity based on both closeness (proximity) AND interconnectivity of clusters, using a graph-partitioning approach. It can find clusters of different shapes, sizes, and densities better than older methods.

9. Density-Based Clustering Methods

Density-based methods form clusters based on the density of data points — a cluster is a region where points are densely packed, separated by regions of low density (noise).

Algorithm	Key Idea
DBSCAN (Density-Based Spatial Clustering of Applications with Noise)	Groups points that are closely packed (have many neighbours within a given radius 'Eps' and minimum points 'MinPts'). Points in low-density regions are marked as noise/outliers. Can find clusters of any shape.
OPTICS (Ordering Points To Identify Clustering Structure)	An improvement over DBSCAN. It does not fix a single density threshold; instead it creates an ordering of points that reveals clusters of varying densities.

10. Grid-Based Clustering Methods

Grid-based methods divide the data space into a finite number of cells that form a grid structure, and then perform clustering on this grid (very fast since it depends on grid size, not number of data objects).

Algorithm	Key Idea
STING (Statistical Information Grid)	Divides the space into rectangular cells at different levels of resolution, storing statistical information (mean, count, etc.) for each cell in a hierarchical (tree) structure.
CLIQUE (Clustering In QUEst)	Combines grid-based and density-based methods. It divides the space into cells and finds dense cells, particularly useful for high-dimensional data.

11. Model-Based Clustering Method (Statistical Approach)

Model-based methods assume that data is generated using a mixture of underlying probability distributions. The method tries to find the best-fit set of parameters for these distributions to describe the clusters.

- Example: Expectation-Maximization (EM) algorithm, which assumes clusters follow a Gaussian (Normal) distribution and finds the best parameters (mean, variance) for each cluster.

12. Association Rules

Association Rule Mining finds interesting relationships (associations) between items in large data sets — most famously used in Market Basket Analysis.

Association Rule format: $A \rightarrow B$ [Support, Confidence]		
Term	**Meaning**	**Formula**

Support	How frequently the itemset {A, B} appears in all transactions	Support(A→B) = P(A ∪B) = (Transactions with both A & B) / (Total transactions)
Confidence	How often B appears in transactions that contain A	Confidence(A→B) = P(B|A) = (Transactions with both A & B) / (Transactions with A)

Example

Rule: {Bread} → {Butter} [Support = 40%, Confidence = 70%] means: 40% of all transactions contain both Bread and Butter; and 70% of the customers who buy Bread also buy Butter.

Large Itemsets and the Basic Algorithm

A Large (Frequent) Itemset is a set of items whose support is greater than or equal to a minimum support threshold set by the user.

Apriori Algorithm (the most important basic algorithm):

- Uses the 'Apriori property': if an itemset is frequent, then ALL of its subsets must also be frequent. (Reverse: if a subset is not frequent, the bigger set is also not frequent — this is used to prune/cut down the search.)
- Step 1: Find all frequent 1-itemsets (single items meeting minimum support).
- Step 2: Use frequent (k-1)-itemsets to generate candidate k-itemsets (join step).
- Step 3: Prune candidates that have any infrequent subset.
- Step 4: Scan the database to count support of candidates and keep only frequent ones.
- Repeat until no more frequent itemsets can be found. Then generate rules meeting minimum confidence.

Parallel and Distributed Algorithms

For very large databases, Apriori is parallelized so multiple processors/machines work together — this splits either the data (Data Distribution / Count Distribution approach) or the candidate itemsets (Candidate Distribution approach) among processors to speed up mining.

Neural Network Approach

Neural Networks can also be used for association rule mining and classification by learning patterns through a network of interconnected nodes (like the human brain), adjusting internal 'weights' during training to recognise relationships between items/attributes.

UNIT V : OLAP, Data Visualization and Warehousing Applications

SYLLABUS — What this unit covers

- Data Visualization and Overall Perspective: Aggregation, Historical Information, Query Facility
- OLAP Functions and Tools
- OLAP Servers: ROLAP, MOLAP, HOLAP
- Data Mining Interface
- Security, Backup and Recovery
- Tuning and Testing the Data Warehouse
- Warehousing Applications and Recent Trends: Web Mining, Spatial Mining, Temporal Mining

1. OLAP — Functions and Tools

OLAP (Online Analytical Processing) is a set of tools/technology that lets users interactively analyse multi-dimensional data from many perspectives, quickly and flexibly (recall Roll-up, Drill-down, Slice, Dice, Pivot from Unit 1).

- OLAP tools help managers ask 'what-if' style analytical questions and get fast answers.
- They work on top of the multi-dimensional data model (data cubes).

2. OLAP Servers — ROLAP, MOLAP, HOLAP

An OLAP server is the engine that sits between the data warehouse and the analysis tools, and actually performs the multi-dimensional analysis.

Type	Full Form	How it Stores Data	Pros	Cons
ROLAP	Relational OLAP	Data stays in relational tables (Star/Snowflake schema); cubes are created on the fly using SQL	Handles very large data; no separate cube storage needed	Slower query response (SQL joins are heavier)
MOLAP	Multi-dimensional OLAP	Data is pre-computed and stored in a special multi-dimensional cube structure	Very fast query response (pre-calculated)	Cube can become very large; less scalable for huge/sparse data
HOLAP	Hybrid OLAP	Combines both — summary data in MOLAP cubes, detailed data in relational (ROLAP) tables	Balances speed and scalability	More complex to design and manage

3. Data Visualization and Overall Perspective

(a) Aggregation

Aggregation means storing summarized data (like totals, averages) rather than only raw detailed data, so common queries (like 'total sales per month') run much faster without scanning every single record.

(b) Historical Information

A data warehouse always keeps historical data (going back several years), since a key feature (Time-Variant, recall Unit 1) is the ability to analyse trends over time, not just the current snapshot.

(c) Query Facility

Data warehouses provide user-friendly query facilities (via OLAP tools, reporting tools, and dashboards) so that even non-technical business users can ask analytical questions without writing complex SQL.

4. Data Mining Interface

The Data Mining Interface is the layer that connects data mining algorithms with the data warehouse, so mining tools can directly pull data for tasks like classification, clustering, and association rule mining, and present the mined patterns back to the user in an easy-to-understand form (charts, rules, trees).

5. Security, Backup and Recovery

Security

- Since the warehouse contains sensitive business data, access control (user login, roles, permissions) is essential — different users should see only what they are authorized to see.
- Data should also be encrypted and audited (tracking who accessed what and when).

Backup and Recovery

- Regular Backups are taken (full, incremental, or differential) so data is not lost if hardware fails.
- Recovery procedures ensure the warehouse can be restored quickly to a consistent state after a failure or crash.

6. Tuning and Testing the Data Warehouse

Tuning

Tuning means improving the performance of the warehouse — for example, using proper indexing (bitmap index), partitioning large tables, creating materialized views for common queries, and optimizing the ETL jobs to run faster.

Testing

Testing checks whether the warehouse works correctly and performs well. Common test types:

Test Type	Purpose
Data Completeness Testing	Checks whether all expected data has been loaded correctly

Test Type	Purpose
Data Accuracy Testing	Checks whether the transformed/loaded data is correct compared to the source
Performance Testing	Checks query response time under normal and heavy load
Integration Testing	Checks whether ETL and reporting tools work together correctly

7. Warehousing Applications and Recent Trends

Data warehousing and mining are used across many industries — banking (fraud detection), retail (market basket analysis), telecom (churn prediction), healthcare (patient pattern analysis), and more.

Recent Trends / Special Types of Mining:

Type	Description	Example
Web Mining	Applying data mining techniques to discover patterns from Web data (web pages, links, usage logs).	Web Content Mining (page text/images), Web Structure Mining (hyperlinks), Web Usage Mining (click/log data)
Spatial Mining	Discovering interesting patterns from spatial (location/geographic) data such as maps and satellite images.	Finding regions with high crime rate, or land-use pattern analysis from satellite images
Temporal Mining	Discovering patterns from time-ordered (sequential) data, where the order/timing of events matters.	Finding patterns like 'stock price rises after a specific news event', sequence-of-purchase analysis